Oriented object detection

Oriented object detection
1. Enable the maximum performance of the mainboard
    1.1. Enable the MAX power mode
    1.2. Enable the Jetson clocks
2. Oriented object detection: the image
        Preview of the effect
3. Oriented object detection: the video
        Preview of the effect
4. Oriented object detection: the real time detection
    4.1. USB camera
        Preview of the effect
References

Use Python to demonstrate **Ultralytics**: the effect of Oriented Bounding Boxes Object Detection on the images, the videos and the real time detection.

1. Enable the maximum performance of the mainboard

1.1. Enable the MAX power mode

Enabling the MAX Power Mode (the maximum power mode) on the Jetson ensures that all the CPU and GPU cores are turned on:

```
sudo nvpmodel -m 0
```

1.2. Enable the Jetson clocks

Enabling the Jetson Clocks ensures that all the CPU and GPU cores run at the maximum frequency:

```
sudo jetson_clocks
```

2. Oriented object detection: the image

Use yolo26n-obb.pt to predict the pictures of the ultralytics project (not the pictures that come with ultralytics).

Enter the code folder:

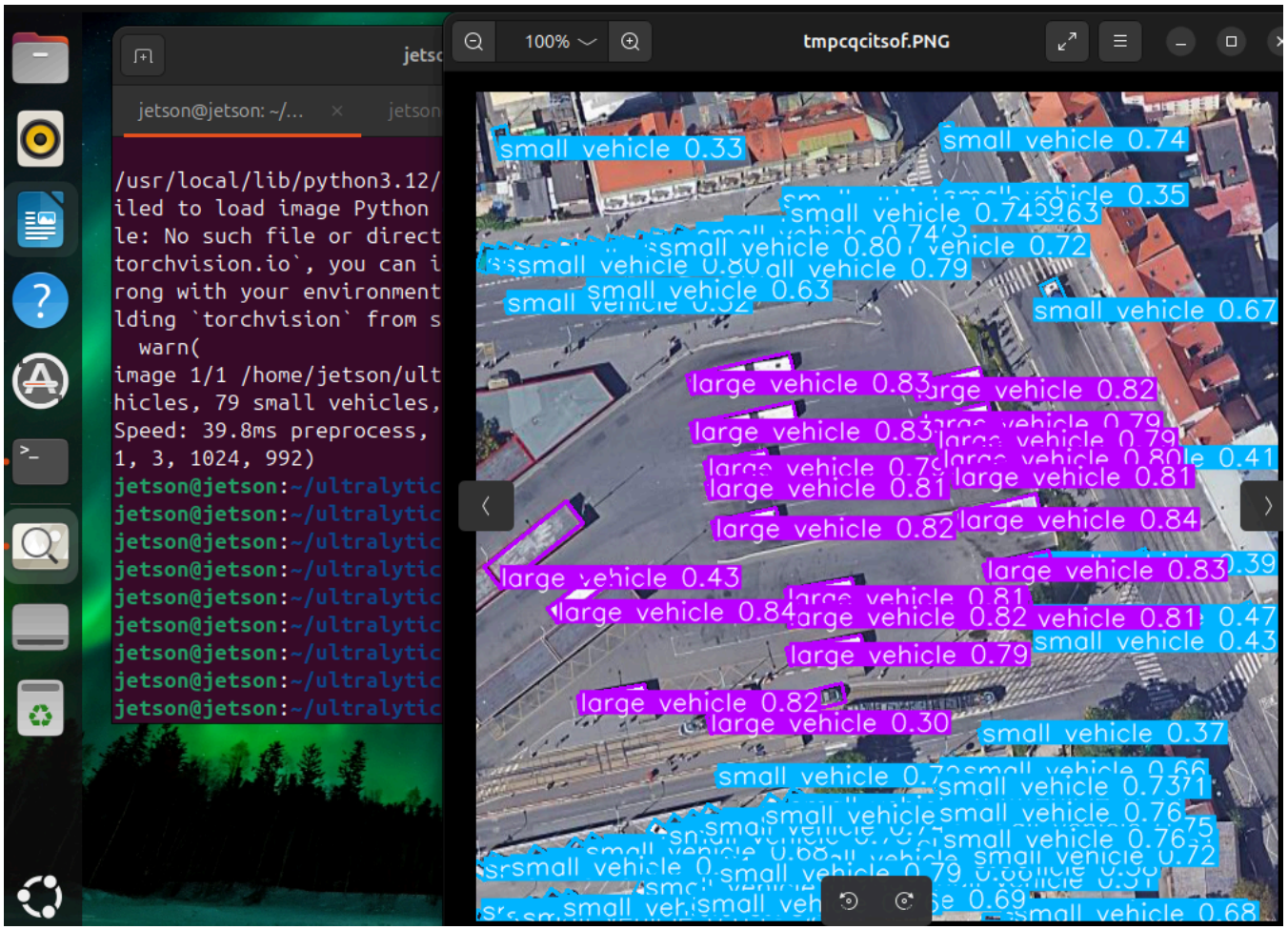
```
cd /home/jetson/ultralytics/ultralytics/yahboom_yolo26
```

Run the code:

```
python3 05.obb_image.py
```

Preview of the effect

The location of the picture that yolo outputs: /home/jetson/ultralytics/ultralytics/output/



The sample code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/jetson/ultralytics/ultralytics/yolo26n-obb.pt")

# Run batched inference on a list of images
results = model("/home/jetson/ultralytics/ultralytics/assets/car.jpg")  # return a list of
Results objects

# Process results list
for result in results:
    # boxes = result.boxes  # Boxes object for bounding box outputs
    # masks = result.masks  # Masks object for segmentation masks outputs
    # keypoints = result.keypoints  # Keypoints object for pose outputs
    # probs = result.probs  # Probs object for classification outputs
    obb = result.obb  # Oriented boxes object for OBB outputs
    result.show()  # display to screen
    result.save(filename="/home/jetson/ultralytics/ultralytics/output/car_output.jpg")  #
save to disk
```

3. Oriented object detection: the video

Use yolo26n-obb.pt to predict a video of the ultralytics project (not the video that comes with ultralytics).

Enter the code folder:

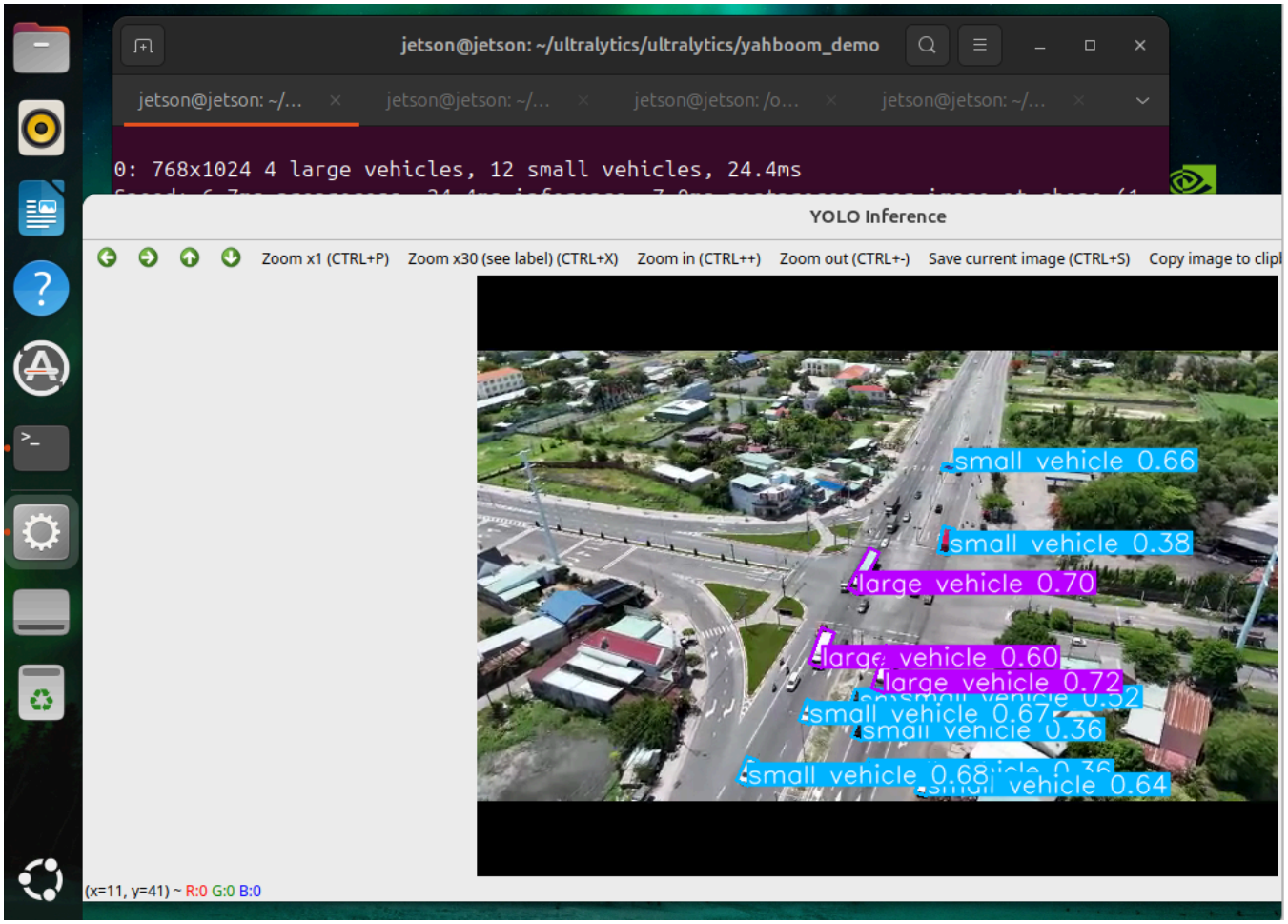
```
cd /home/jetson/ultralytics/ultralytics/yahboom_yolo26
```

Run the code:

```
python3 05.obb_video.py
```

Preview of the effect

The location of the video that yolo outputs: /home/jetson/ultralytics/ultralytics/output/



The sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/ultralytics/yolo26n-obb.pt")

# Open the video file
video_path = "/home/jetson/ultralytics/ultralytics/videos/street.mp4"
```

```
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/jetson/ultralytics/ultralytics/output/05.street_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v' depending on your
platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

4. Oriented object detection: the real time detection

4.1. USB camera

Use yolo26n-obb.pt to predict the image of the USB camera.

Enter the code folder:

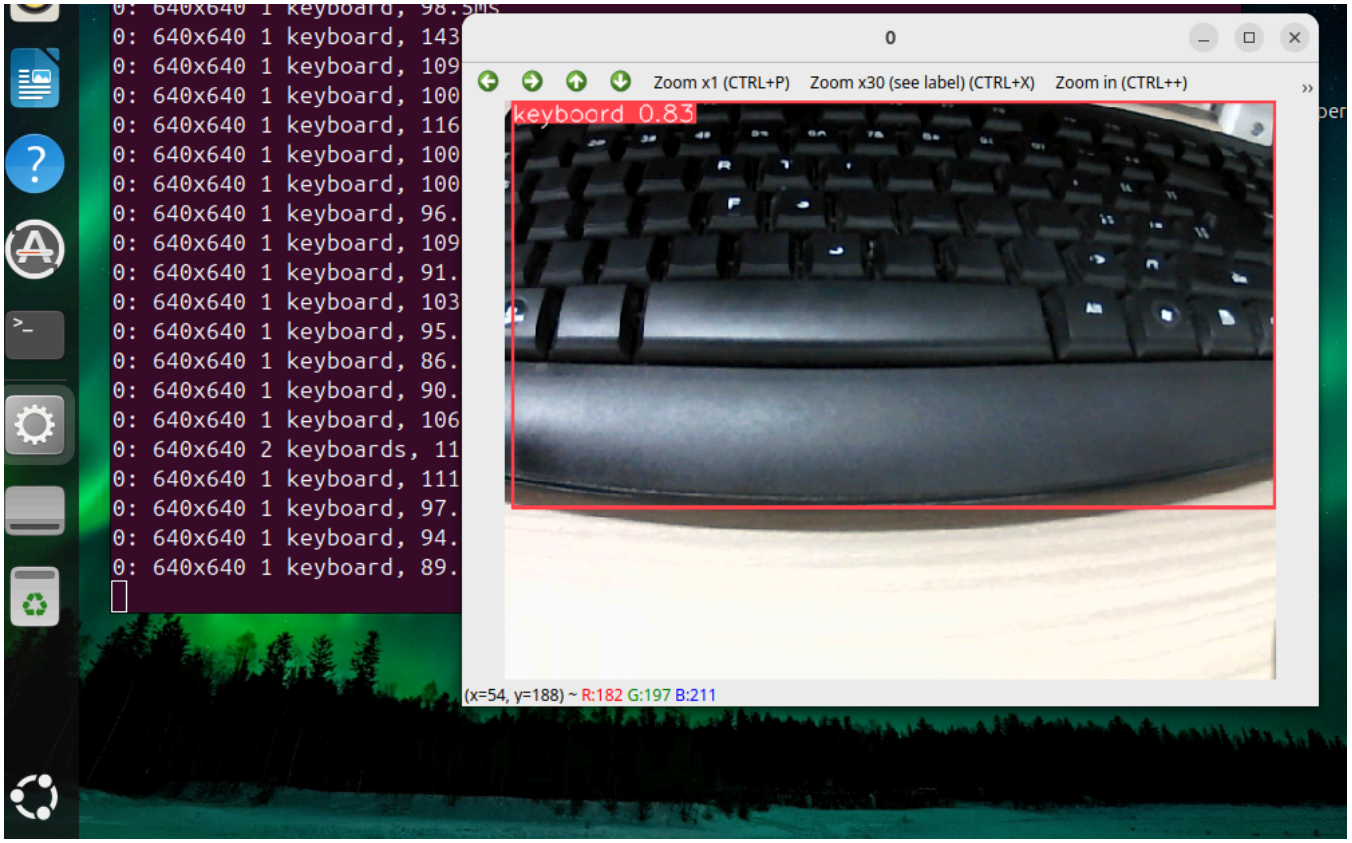
```
cd /home/jetson/ultralytics/ultralytics/yahboom_yolo26
```

Run the code: click the preview window and press the q key to terminate the program!

```
python3 05.obb_camera_usb.py
```

Preview of the effect

The location of the video that yolo outputs: /home/jetson/ultralytics/ultralytics/output/



The sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/ultralytics/yolo26n-obb.pt")

# Open the cammera
cap = cv2.VideoCapture(0)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/jetson/ultralytics/ultralytics/output/05.obb_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v' depending on your
platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))
```

```
# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

References

https://docs.ultralytics.com/tasks/obb/